Design and Implementation of a Lightweight Multidimensional Tensor Library in C++

Abstract

Multidimensional arrays, or tensors, form the computational bedrock of contemporary numerical analysis, scientific simulations, computer vision, and machine learning systems. While high-level environments provide rich array abstractions, systems-level C++ software frequently struggles with the dichotomy between heavy, complex third-party frameworks and fragile, cache-inefficient nested pointer structures such as `std::vector<std::vector<T>>`.

This paper presents the design, mathematical foundations, architectural implementation, empirical evaluation, and complete user documentation of **MiniTensor**, an open-source, dependency-free, header-only C++20 library for arbitrary-rank multidimensional tensors. MiniTensor couples an owning, contiguous memory architecture with dynamic metadata separation, row-major stride indexing, checked dimensional arithmetic, and cache-conscious linear algebra kernels.

We discuss the design decisions underpinning its resource acquisition is initialization (RAII) memory model, bounds-checked offset engine, integer overflow safeguards, and cache-tiled and OpenMP-accelerated matrix and batched tensor multiplication implementations. In version 1.1.0, MiniTensor generalized tensor multiplication across arbitrary ranks, including 1D vector dot products, 2D matrix multiplication, higher-rank batched contractions, batch broadcasting, and arbitrary-axis tensor contractions (`tensordot`).

Experimental evaluation conducted on modern x86-64 hardware demonstrates that MiniTensor's contiguous storage and loop reordering achieve up to a 5.49x parallel speedup over baseline execution on multi-core workloads, while maintaining strict mathematical precision, memory safety, and zero external dependencies. The paper concludes with an extensive practical user manual and API reference guide for seamless adoption.

Table of Contents

1. Introduction
2. Theoretical Background and Foundations
   * 2.1 Tensor Definitions and Dimensional Ranks
   * 2.2 Memory Linearization and Row-Major Ordering
   * 2.3 Strided Coordinate Mapping
   * 2.4 Algebraic Operations: Element-Wise vs. Linear Algebra
   * 2.5 Geometric Transformations: Reshape and Transposition
3. Related Work and Design Space Analysis
   * 3.1 Native Multidimensional Arrays and Nested Vectors
   * 3.2 Template-Heavy Linear Algebra Frameworks (Eigen)
   * 3.3 Modern Expression Template Libraries (xtensor)
   * 3.4 NumPy as an Architectural Reference
   * 3.5 MiniTensor's Position in the Design Space
4. System Architecture and Design
   * 4.1 Structural Modularity and Header Distribution
   * 4.2 The `Tensor<T>` Class Template
   * 4.3 Metadata and Storage Separation
   * 4.4 Memory Ownership Model
   * 4.5 Public Interface and Ergonomics
5. Implementation Details
   * 5.1 Safe Shape Products and Overflow Prevention
   * 5.2 Stride Computation and Empty Tensor Handling
   * 5.3 Multidimensional Indexing and Coordinate Validation
   * 5.4 Element-Wise Operations Engine
   * 5.5 Arbitrary-Rank Multiplication Kernels: Serial, Blocked, and Parallel
   * 5.6 Materialized Reshaping and Transposition
   * 5.7 Algorithmic Complexity Summary
6. Memory Management and Hardware Co-Design
   * 6.1 Contiguous Storage and Modern CPU Cache Hierarchies
   * 6.2 Spatial Locality and Cache Line Utilization
   * 6.3 The Cache-Optimal `i-k-j` Loop Ordering
   * 6.4 Sub-Block Cache Tiling
   * 6.5 Thread Scheduling and Memory Bus Contention
7. Experimental Evaluation
   * 7.1 Experimental Testbed and Configuration

* 7.2 Benchmark Methodology and Measurement Protocol
* 7.3 Empirical Measurements
* 7.4 Analysis of Performance and Scalability
8. Practical Applications
* 8.1 Digital Image Processing and Computer Vision
* 8.2 Dense Neural Network Layers
* 8.3 Grid-Based Numerical Simulations
9. Limitations and Future Research Directions
* 9.1 Current Architectural Constraints
* 9.2 Proposed Research Directions
10. Conclusion
11. References
12. Comprehensive User Documentation and Integration Guide
* 12.1 Distribution and Download Options
* 12.2 Integration and Build Setup
* 12.3 Complete Multi-Rank Code Walkthrough
* 12.4 Compiler Commands and Flags
* 12.5 API Reference and Method Cheat Sheet

1. Introduction

The manipulation of multidimensional discrete data is central to modern scientific computation. Fields ranging from computational fluid dynamics and quantum mechanics to digital signal processing, medical imaging, and deep learning depend on operations performed over structured N-dimensional grids [1], [2]. In mathematical terminology, these multidimensional objects are classified as **tensors**.

In systems programming languages such as C++, managing multidimensional data presents significant engineering trade-offs. The standard library provides contiguous linear collections, such as `std::vector<T>`, but lacks native abstractions for higher-dimensional indexing, shape manipulation, and linear algebra. Consequently, developers frequently adopt one of two suboptimal patterns:

1. **Ad-hoc nested containers**, such as `std::vector<std::vector<T>>`, which introduce pointer-chasing overhead, severe memory fragmentation, and cache misses [3].
2. **Monolithic industrial frameworks**, such as Eigen, Armadillo, or PyTorch's LibTorch, which impose substantial compilation overhead, extensive dependency trees, steep learning curves, and complex template metaprogramming semantics [4], [5].

To address the need for a transparent, self-contained, and performant tensor abstraction, this paper introduces **MiniTensor**. MiniTensor is an open-source, dependency-free C++20 library implementing arbitrary-rank, contiguous, owning multidimensional tensors.

The primary objectives of the MiniTensor project are:

* To provide an intuitive, mathematically sound N-dimensional tensor container with dynamic runtime rank and shape capabilities.
* To guarantee memory safety, exception safety, and deterministic resource deallocation via the Resource Acquisition Is Initialization (RAII) paradigm.
* To design cache-cooperative computational kernels that maximize memory bandwidth and spatial locality on modern CPU architectures.
* To maintain an accessible, header-only implementation devoid of external library dependencies.

The remainder of this paper is structured as follows. Section 2 establishes the theoretical and mathematical foundations of tensor computing. Section 3 contextualizes MiniTensor against existing array architectures. Section 4 presents the system architecture. Section 5 details the concrete algorithms and edge-case handling. Section 6 analyzes memory hardware co-design. Section 7 presents empirical benchmark results. Section 8 outlines practical applications. Section 9 discusses limitations and future research, and Section 10 concludes the paper.

2. Theoretical Background and Foundations

2.1 Tensor Definitions and Dimensional Ranks

Mathematically, a tensor T can be characterized as a multilinear mapping over vector spaces. In the context of computer science and numerical computing, a tensor is formalized as a multidimensional array whose entries are indexed by a tuple of discrete coordinates [6].

The **rank** (also termed *order* or *dimensionality*) of a tensor is the number of independent axes required to address an individual element:

$$\mathbf{Rank(T) = K, \quad K \in \mathbb{N}_0}$$

* **Rank 0 (Scalar):** A zero-dimensional tensor representing an isolated numerical value:

$T \in \mathbb{R}$

* **Rank 1 (Vector):** A one-dimensional array indexed by a single discrete coordinate i:

v in R^{D_0}, v_i in R

* **Rank 2 (Matrix):** A two-dimensional grid indexed by row i and column j:
M in R^{D_0 x D_1}, M_{i,j} in R
* **Rank K (General Tensor):** A discrete K-dimensional array indexed by a coordinate tuple (i_0, i_1, ..., i_{K-1}):
T in R^{D_0 x D_1 x ... x D_{K-1}}

The sequence (D_0, D_1, ..., D_{K-1}) constitutes the **shape** of the tensor. The total cardinality N of the tensor elements is given by the product of its dimensional extents:

$$N = \prod_{k=0}^{K-1} D_k$$

For a scalar (K=0), the empty product yields N = 1. For any shape containing a degenerate dimension where D_k = 0, N = 0.

2.2 Memory Linearization and Row-Major Ordering

Physical computer memory (RAM) is organized as a flat, one-dimensional sequence of byte addresses M: N_0 -> Byte. Mapping a multidimensional coordinate space Z^{D_0 x ... x D_{K-1}} onto a contiguous memory interval [0, N-1] requires a systematic serialization convention [7].

MiniTensor adopts **Row-Major Order** (lexicographical ordering), which is the standard convention in C, C++, and Python's NumPy. Under row-major layout, adjacent elements along the last axis (K-1) reside consecutively in physical memory.

```
Logical 2x3 Matrix:
+--------------------+
| (0,0)  (0,1)  (0,2) |  Row 0
| (1,0)  (1,1)  (1,2) |  Row 1
+--------------------+

Physical 1D Memory Layout:
Offset:    0      1      2      3      4      5
Buffer: [ (0,0), (0,1), (0,2), (1,0), (1,1), (1,2) ]
        _______Row 0______/   _______Row 1______/
```

2.3 Strided Coordinate Mapping

To transform an arbitrary multidimensional index vector i = (i_0, i_1, ..., i_{K-1}) into a linear scalar memory offset Offset in [0, N-1], MiniTensor utilizes **strides**.

The stride S_k of axis k defines the displacement in physical memory required to advance the coordinate i_k by 1 unit:

$$S_{K-1} = 1$$

$$S_k = \prod_{m=k+1}^{K-1} D_m = S_{k+1} \times D_{k+1}, \quad \text{for } 0 \le k < K-1$$

Given precomputed strides S = (S_0, S_1, ..., S_{K-1}), the offset function f: N_0^K -> N_0 is defined as:

$$\text{Offset}(i) = \sum_{k=0}^{K-1} i_k * S_k$$

For example, consider a 3D tensor of shape (2, 3, 4):

* S_2 = 1
* S_1 = 1 x 4 = 4
* S_0 = 4 x 3 = 12
* Strides vector: (12, 4, 1).

To resolve coordinate (1, 2, 3):

$$\text{Offset}(1, 2, 3) = (1 \times 12) + (2 \times 4) + (3 \times 1) = 12 + 8 + 3 = 23$$

2.4 Algebraic Operations: Element-Wise vs. Linear Algebra

A foundational principle in tensor software design is the strict mathematical distinction between **element-wise operations** and **linear algebraic contractions**:

1. **Element-Wise Operations (Hadamard Operations):**
Given two tensors A, B in R^{D_0 x ... x D_{K-1}} with identical shapes, binary addition, subtraction, and multiplication are defined point-wise:

(A +/- B)[i_0, ..., i_{K-1}] = A[i_0, ..., i_{K-1}] +/- B[i_0, ..., i_{K-1}]

(A (x) B)[i_0, ..., i_{K-1}] = A[i_0, ..., i_{K-1}] x B[i_0, ..., i_{K-1}]

2. **Multidimensional Linear Contraction and Multiplication:**
Rather than restricting multiplication to 2-dimensional matrices, MiniTensor implements generalized tensor multiplication across arbitrary ranks (K >= 1):

* **Rank 1 x Rank 1 (Vector Dot Product):** Contracts two vectors u, v in R^K into a Rank 0 scalar:

u * v = sum_{k=0}^{K-1} u_k * v_k in R

* **Rank 2 x Rank 1 (Matrix-Vector Contraction):** Contracts A in R^{M x K} with v in R^K producing y in R^M:
y_i = sum_{k=0}^{K-1} A_{i,k} * v_k
* **Rank 1 x Rank 2 (Vector-Matrix Contraction):** Contracts u in R^K with B in R^{K x N} producing y in R^N:
y_j = sum_{k=0}^{K-1} u_k * B_{k,j}
* **Rank 2 x Rank 2 (Standard Matrix Multiplication):** Contracts inner dimension K across M x K and K x N matrices:
C_{i,j} = sum_{k=0}^{K-1} A_{i,k} * B_{k,j}
* **Rank >= 3 (Batched Matrix Multiplication):** For tensors sharing arbitrary leading batch dimensions (B_0, ..., B_{p-1}), contractions execute independently across each slice:
C_{b_0, ..., b_{p-1}, i, j} = sum_{k=0}^{K-1} A_{b_0, ..., b_{p-1}, i, k} * B_{b_0, ..., b_{p-1}, k, j}
* **Batch Broadcasting:** When a batched tensor (B_0, ..., B_{p-1}, M, K) is multiplied with a unbatched matrix (K, N), the matrix is broadcast across all batch slices yielding (B_0, ..., B_{p-1}, M, N).
* **Arbitrary-Axis Contraction (`tensordot`):** Contracts arbitrary axes between two general tensors by summing products over specified axis dimensions, matching the semantics of general Einstein tensor summation.

2.5 Geometric Transformations: Reshape and Transposition

1. **Reshape:** Reinterprets the tensor's shape from (D_0, ..., D_{K-1}) to (D'_0, ..., D'_{M-1}) under the invariant of element conservation:
prod_{k=0}^{K-1} D_k = prod_{m=0}^{M-1} D'_m
2. **Transposition:** For a matrix A in R^{M x N}, the transposed matrix A^T in R^{N x M} reflects elements across the principal diagonal:
A^T_{j,i} = A_{i,j}

---

3. Related Work and Design Space Analysis

The design of numerical array software occupies a broad spectrum of trade-offs between performance, compilation complexity, abstraction overhead, and external dependencies.

3.1 Native Multidimensional Arrays and Nested Vectors

Native C-style arrays (`T arr[M][N]`) provide contiguous storage but require dimension bounds to be known at compile time, preventing dynamic resizing and runtime rank manipulation.

The standard C++ idiom for dynamic multidimensional tables is nesting vectors: `std::vector<std::vector<T>>`. While intuitive, this approach exhibits severe architectural deficiencies:

* **Pointer Chasing:** The outer vector stores an array of `std::vector` control objects. Accessing an element `vec[i][j]` requires dereferencing an outer pointer to find the inner heap buffer, stalling the CPU instruction pipeline [3].
* **Cache Inefficiency:** Each row is allocated independently on the heap. Spatial proximity between consecutive rows is non-existent, causing frequent cache misses during vertical or block traversals.
* **Memory Overhead:** Each vector object incurs overhead for three internal pointers (typically 24 bytes on 64-bit platforms) plus heap allocator metadata.
* **Ragged Geometry:** Nothing enforces rectangularity; row sizes can vary arbitrarily, necessitating defensive runtime checks.

3.2 Template-Heavy Linear Algebra Frameworks (Eigen)

Eigen is a mature C++ library for linear algebra [4]. It relies heavily on expression templates to eliminate intermediate copies during compound operations (e.g., A + B + C).

However, Eigen's sophisticated metaprogramming leads to substantial compile-time overhead, verbose error diagnostics, and distinct types for compile-time vs. runtime sizing (`Matrix3f` vs. `MatrixXf`). For embedded systems, educational settings, or lightweight applications requiring transparent code, Eigen can introduce unnecessary complexity.

3.3 Modern Expression Template Libraries (xtensor)

`xtensor` offers an N-dimensional array container in C++ inspired by NumPy [5]. It features lazy evaluation, multidimensional broadcasting, and non-owning views.

While expressive, `xtensor` depends on external libraries (such as `xsimd`) for vectorization and utilizes intricate template hierarchies that increase build duration and cognitive load for non-expert C++ developers.

3.4 NumPy as an Architectural Reference

NumPy's `ndarray` architecture serves as the standard computational model for strided arrays [8]. NumPy maintains an underlying flat data buffer accompanied by shape and stride metadata. MiniTensor adapts this core design principle into modern, type-safe C++20.

3.5 MiniTensor's Position in the Design Space

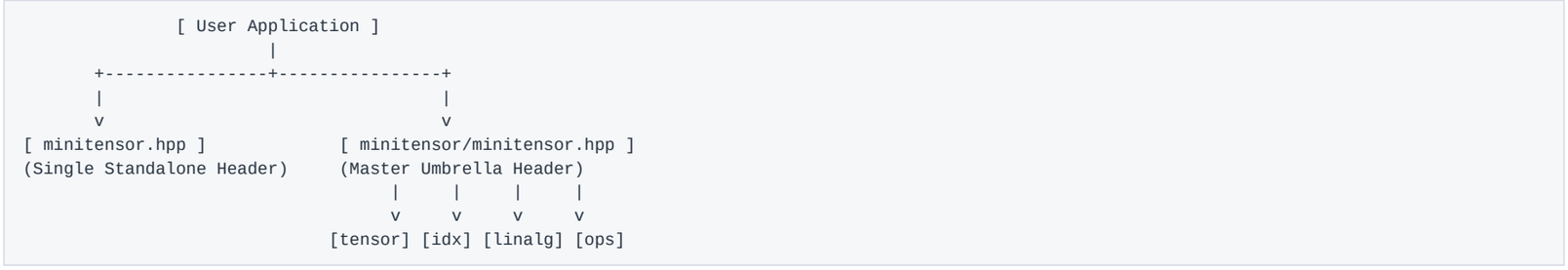
MiniTensor occupies a distinct niche in the design space:

* **Dependency-Free & Header-Only:** Requires only a standard C++20 compiler.
* **Strict Owning Value Semantics:** Eliminates dangling reference hazards by treating tensors as value types.
* **Algorithmic Transparency:** High-performance kernels (`i-k-j` matrix multiplication, cache tiling, OpenMP) implemented clearly without obfuscating template layers.
* **Robust Exception Guarantees:** All edge cases (overflow, dimension mismatches, out-of-bounds indices) are checked and surfaced via standard C++ exceptions.

4. System Architecture and Design

4.1 Structural Modularity and Header Distribution

MiniTensor is architected with dual distribution channels: a modular multi-header architecture for deep integration and a unified single-header file for rapid adoption:



* **Standalone Single Header (`release/minitensor.hpp`):** Complete, self-contained single-header file (~30 KB). Zero external dependencies, drop-in ready for fast compilation.
* **Master Umbrella Header (`include/minitensor/minitensor.hpp`):** Aggregates all modular headers.
* **Modular Component Headers:**
* `tensor.hpp`: Core definition of the `Tensor<T>` class template.
* `indexing.hpp`: Stride mathematics, dimensional offsets, and bounds-checking primitives.
* `linalg.hpp`: Arbitrary-rank tensor multiplication (`matmul`, `matmul_blocked`, `matmul_parallel`) and contraction routines (`tensordot`).
* `operations.hpp`: Free algebraic functions and element-wise utilities.

4.2 The `Tensor<T>` Class Template

The entire tensor abstraction is parameterized over type `T`:

```
namespace minitensor {
template <typename T>
class Tensor {
    // ...
private:
    std::vector<std::size_t> shape_;
    std::vector<std::size_t> strides_;
    std::vector<T> data_;
};
}
```

By parameterizing on `T`, the library supports arbitrary arithmetic types (`float`, `double`, `int32_t`, `uint8_t`) with zero runtime polymorphism overhead.

4.3 Metadata and Storage Separation

A core design tenet of MiniTensor is the complete separation between structural metadata and memory storage:

* **Metadata:** Two dynamic arrays of type `std::vector<std::size_t>`:
* `shape_`: Defines the extent along each axis.
* `strides_`: Stores the precomputed coordinate displacement factors.
* **Storage:** A single contiguous `std::vector<T>` containing all N = PROD D_k elements.

```
Stack Frame: minitensor::Tensor<T>
+-------------------------------------------------------------+
| shape_   = std::vector<size_t> { M, N }                     |
| strides_ = std::vector<size_t> { N, 1 }                     |
| data_    = std::vector<T> ---------------------------------+ |
+-------------------------------------------------------------|-+
                                                              |
Heap Allocation (Single Contiguous Buffer):                   v
+---------------+---------------+---------------+-------------+--
| Element (0,0) | Element (0,1) | Element (0,2) | Element(1,0)| ...
+---------------+---------------+---------------+-------------+--
```

4.4 Memory Ownership Model

MiniTensor enforces **exclusive value ownership** governed by standard RAII semantics:

* **No Shared Buffers:** Unlike NumPy or PyTorch, MiniTensor does not provide aliased, non-owning tensor views. Every tensor owns its underlying storage buffer.
* **Deep Copies:** Copy construction and copy assignment allocate a new heap buffer and duplicate all elements. Modifications to a copy are guaranteed never to affect the source.
* **Move Semantics:** Move construction and move assignment execute in O(1) time by transferring vector pointers without heap reallocations.

4.5 Public Interface and Ergonomics

Element access is designed to balance safety, performance, and ergonomic syntax:

* `at(std::initializer_list<std::size_t>)` and `at(std::vector<std::size_t>)`: Fully bounds-checked coordinate access returning mutable or const references.
* `set(indices, value)`: Explicit coordinate assignment.
* `operator()(indices...)`: Variadic call operator constrained via C++20 concepts (`std::is_convertible_v<Indices, std::size_t> && ...`) enabling natural mathematical syntax:

```cpp
tensor(row, col) = 42.0f;
```

* `operator()()`: Dedicated scalar accessor for rank-0 tensors.

5. Implementation Details

5.1 Safe Shape Products and Overflow Prevention

On modern 64-bit computing architectures, computing the total size of a high-dimensional tensor risks integer multiplication overflow if dimension values are excessively large. An undetected integer wrap-around could cause the runtime to allocate a small buffer, resulting in heap corruption upon element traversal.

MiniTensor prevents this vulnerability via checked arithmetic in `calculate_size()`:

```
static std::size_t calculate_size(const std::vector<std::size_t>& shape) {
    for (const std::size_t dimension : shape) {
        if (dimension == 0) {
            return 0;
        }
    }

    std::size_t total_size = 1;
    for (const std::size_t dimension : shape) {
        if (total_size > std::numeric_limits<std::size_t>::max() / dimension) {
            throw std::length_error("Tensor shape product overflows size_t");
        }
        total_size *= dimension;
    }
```

```
    return total_size;
}
```

If the product of dimensions exceeds `SIZE_MAX`, the library immediately throws `std::length_error` before any memory is allocated. Furthermore, if any dimension is zero, the function returns 0, correctly identifying empty tensors.

5.2 Stride Computation and Empty Tensor Handling

Strides are calculated during tensor initialization in `calculate_strides()`:

```
static std::vector<std::size_t> calculate_strides(
    const std::vector<std::size_t>& shape) {
    static_cast<void>(calculate_size(shape));
    std::vector<std::size_t> strides(shape.size(), 0);
    if (std::find(shape.begin(), shape.end(), 0) != shape.end()) {
        return strides;
    }

    std::size_t stride = 1;
    for (std::size_t axis = shape.size(); axis > 0; --axis) {
        strides[axis - 1] = stride;
        if (stride > std::numeric_limits<std::size_t>::max() / shape[axis - 1]) {
            throw std::length_error("Tensor stride overflows size_t");
        }
        stride *= shape[axis - 1];
    }
    return strides;
}
```

Notice the handling of empty tensors: if any dimension is zero, `strides` is populated with zeros (`{0, 0, ...}`). This eliminates undefined offset multiplications for degenerate tensors.

5.3 Multidimensional Indexing and Coordinate Validation

Coordinate lookup is executed by the private helper `calculate_offset()`:

```
[[nodiscard]] std::size_t calculate_offset(
    const std::vector<std::size_t>& indices) const {
    if (indices.size() != rank()) {
        throw std::invalid_argument("Number of indices must match tensor rank");
    }

    std::size_t offset = 0;
    for (std::size_t axis = 0; axis < rank(); ++axis) {
        if (indices[axis] >= shape_[axis]) {
            throw std::out_of_range("Tensor index is out of bounds");
        }
        offset += indices[axis] * strides_[axis];
    }
    return offset;
}
```

Access safety is enforced across two dimensions:

1. **Rank Equality:** Providing fewer or more indices than `rank()` throws `std::invalid_argument`.
2. **Dimension Boundary:** Supplying an index exceeding `shape_[axis]` throws `std::out_of_range`.

5.4 Element-Wise Operations Engine

Binary element-wise arithmetic operators (`+`, `-`, `*`) delegate to a unified private template engine:

```
template <typename Operation>
[[nodiscard]] Tensor elementwise(const Tensor& other, Operation operation) const {
    if (shape_ != other.shape_) {
        throw std::invalid_argument(
            "Element-wise operations require identical tensor shapes");
```

```
    }

    std::vector<T> result_data;
    result_data.reserve(size());
    for (std::size_t index = 0; index < size(); ++index) {
        result_data.push_back(operation(data_[index], other.data_[index]));
    }
    return Tensor(shape_, std::move(result_data));
}
```

Because MiniTensor requires identical shapes and maintains contiguous row-major layouts, element-wise operations operate as flat 1D loops over linear buffers. This permits modern compilers to auto-vectorize the loop using SIMD vector instructions (AVX2 / SSE / NEON).

5.5 Matrix and Tensor Multiplication: Arbitrary-Rank and Batched Kernels

MiniTensor provides generalized tensor multiplication supporting operations across arbitrary ranks:

* **Vector Dot Product (Rank 1 x Rank 1 -> Rank 0):** Inner scalar contraction: sum_k A_k B_k.
* **Matrix-Vector Product (Rank 2 x Rank 1 -> Rank 1):** sum_k A_{m, k} B_k.
* **Vector-Matrix Product (Rank 1 x Rank 2 -> Rank 1):** sum_k A_k B_{k, n}.
* **Matrix Multiplication (Rank 2 x Rank 2 -> Rank 2):** Standard (M x K) x (K x N) -> (M x N).
* **Batched Matrix Multiplication (Rank >= 3):** Leading batch dimensions are aligned and broadcast:

(B_0, ..., B_{p-1}, M, K) x (B_0, ..., B_{p-1}, K, N) -> (B_0, ..., B_{p-1}, M, N)

* **Arbitrary Axis Contraction (`tensordot`):** General tensor contraction across arbitrary paired axes.

1. Baseline Serial Kernel (`matmul`)

Executes across all batch slices using an optimized `i-k-j` loop traversal:

```
for (std::size_t b = 0; b < plan.total_batches; ++b) {
    // Resolve batch offsets via precomputed batch strides...
    for (std::size_t row = 0; row < plan.rows; ++row) {
        for (std::size_t inner = 0; inner < plan.inner; ++inner) {
            const T left = data_[a_batch_off + row * a_row_stride + inner * a_inner_stride];
            for (std::size_t column = 0; column < plan.columns; ++column) {
                result.data_[res_batch_off + row * plan.columns + column] +=
                    left * other.data_[b_batch_off + inner * b_inner_stride + column * b_col_stride];
            }
        }
    }
}
```

2. Cache-Blocked Kernel (`matmul_blocked`)

Tiles both the row dimension and the inner contraction dimension into sub-blocks of size B (default 64) across all batch slices:

```
for (std::size_t b = 0; b < plan.total_batches; ++b) {
    for (std::size_t row_block = 0; row_block < plan.rows;) {
        const std::size_t row_end = row_block + std::min(block_size, plan.rows - row_block);
        for (std::size_t inner_block = 0; inner_block < plan.inner;) {
            const std::size_t inner_end = inner_block + std::min(block_size, plan.inner - inner_block);
            for (std::size_t row = row_block; row < row_end; ++row) {
                for (std::size_t inner = inner_block; inner < inner_end; ++inner) {
                    const T left = data_[a_batch_off + row * a_row_stride + inner * a_inner_stride];
                    for (std::size_t column = 0; column < plan.columns; ++column) {
                        result.data_[res_batch_off + row * plan.columns + column] +=
                            left * other.data_[b_batch_off + inner * b_inner_stride + column * b_col_s...
                    }
                }
            }
            inner_block = inner_end;
        }
        row_block = row_end;
    }
}
```

3. OpenMP Parallel Kernel (`matmul_parallel`)

Parallelizes across batches (when multiple batches exist) or across matrix rows (for single large matrices) using OpenMP:

```
#pragma omp parallel for schedule(static)
for (std::ptrdiff_t b = 0; b < static_cast<std::ptrdiff_t>(plan.total_batches); ++b) {
    // Process batch slice b without write contention...
}
```

5.6 Materialized Reshaping and Transposition

* **`reshape(new_shape)`:** Validates that PROD D'_m == N. It constructs a new owning `Tensor<T>` by copying the internal vector buffer.
* **`transpose()`:** Validates that `rank() == 2`. It allocates a matrix of shape (D_1, D_0) and maps elements via:

result[col x D_0 + row] = data[row x D_1 + col]

5.7 Algorithmic Complexity Summary

Operation	Mathematical Form	Operands / Rank	Time Complexity	Extra Space Complexity
Tensor Addition	A + B	Rank K >= 0	O(N)	O(N)
Tensor Subtraction	A - B	Rank K >= 0	O(N)	O(N)
Hadamard Multiplication	A (x) B	Rank K >= 0	O(N)	O(N)
Vector Dot Product	u * v	Rank 1 x Rank 1	O(K)	O(1)
Matrix-Vector Contraction	A v	Rank 2 x Rank 1	O(M * K)	O(M)
Matrix Multiplication	A B	Rank 2 x Rank 2	O(M * K * N)	O(M * N)
Batched Matrix Mul	A x B	Rank >= 3 (B slices)	O(B * M * K * N)	O(B * M * N)
Blocked Matrix Mul	A x B	Rank >= 2 (B slices)	O(B * M * K * N)	O(B * M * N)
Parallel Matrix Mul	A x B	Rank >= 2 (P threads)	O(frac{B * M * K * N}{P}..	O(B * M * N)
Tensor Contraction (`ten..	A x B	General Ranks	O(V * K_{contract})	O(V)
Reshape	A -> A'	Rank K -> M	O(N)	O(N)
Matrix Transpose	A -> A^T	Rank 2	O(M * N)	O(M * N)

** Where P represents the number of active OpenMP worker threads, B is total batch count, and V is total output volume.*

6. Memory Management and Hardware Co-Design

6.1 Contiguous Storage and Modern CPU Cache Hierarchies

Contemporary microprocessors operate in a multi-tiered memory hierarchy where latency increases exponentially with distance from the CPU register file [9]:

* **Registers:** ~= 0.5 ns latency.
* **L1 Data Cache (32-48 KB/core):** ~= 1 ns latency.
* **L2 Cache (512-1024 KB/core):** ~= 3-5 ns latency.
* **L3 Cache (Shared, 16-64 MB):** ~= 15-20 ns latency.
* **Main Memory (DRAM):** ~= 60-100 ns latency.

Because arithmetic execution units can execute multiple instructions per nanosecond, an algorithm that accesses DRAM randomly or discontinuously spends over 90% of its execution cycles stalled awaiting memory fetches.

6.2 Spatial Locality and Cache Line Utilization

CPUs transfer data between main memory and caches in discrete granularities known as **Cache Lines** (standardized at 64 bytes on modern x86 and ARM processors).

When a program requests a single 32-bit floating-point value (4 bytes), the memory bus loads an entire 64-byte block containing 16 consecutive float values.

* In **contiguous row-major storage**, sequential operations access elements 0, 1, 2, ..., 15 consecutively. The first access incurs a cache line transfer, but the subsequent 15 accesses achieve instantaneous L1 cache hits.
* In **columnar or strided traversals**, each access jumps across memory by an entire row, requiring a new 64-byte fetch for every single 4-byte float. This wastes frac{15}{16} = 93.75% of the consumed memory bus bandwidth.

MiniTensor's contiguous linear buffer ensures maximal spatial locality for row traversals, tensor copying, and element-wise arithmetic.

6.3 The Cache-Optimal `i-k-j` Loop Ordering

Textbook implementations of matrix multiplication are typically formulated with loop order i -> j -> k:

```
// Classical i-j-k loop ordering:
for i from 0 to M:
    for j from 0 to N:
        for k from 0 to K:
            C[i,j] += A[i,k] * B[k,j]  <-- Column access on B!
```

In the inner loop, incrementing k alters the row index of B, traversing vertically across physical memory by stride N. For matrices larger than L1 cache, this access pattern triggers a cache miss on almost every inner step.

MiniTensor reorders the loops to i -> k -> j:

```
// MiniTensor i-k-j loop ordering:
for i from 0 to M:
    for k from 0 to K:
        left = A[i,k]                 <-- Stored in CPU register!
        for j from 0 to N:
            C[i,j] += left * B[k,j]    <-- Contiguous unit-stride access!
```

Under this reordering:

1. `left` is cached in a scalar CPU register.
2. `B[k, j]` advances with unit stride across row k.
3. `C[i, j]` advances with unit stride across row i.

Both input and output buffers stream through memory sequentially, allowing full cache line exploitation and enabling compilers to auto-vectorize the loop into packed SIMD FMA (Fused Multiply-Add) instructions.

6.4 Sub-Block Cache Tiling

When matrix dimensions exceed 512 x 512, the active working set exceeds the capacity of L2 caches. To maintain high cache residency, MiniTensor's `matmul_blocked` decomposes matrices into B x B blocks (default B = 64).

A 64 x 64 tile of single-precision floats occupies:

$$64 \times 64 \times 4 \text{ bytes} = 16{,}384 \text{ bytes } (16 \text{ KB})$$

Because 16 KB fits comfortably inside standard 32 KB or 48 KB L1 Data Caches, the inner block products execute while the tile data remains resident in L1 cache, minimizing DRAM bus transfers.

6.5 Thread Scheduling and Memory Bus Contention

In `matmul_parallel`, MiniTensor utilizes OpenMP with `#pragma omp parallel for schedule(static)`.

Because output matrix rows are assigned exclusively to individual worker threads, there are no shared writable states, eliminating false sharing, mutex locks, and synchronization contention.

However, as matrix dimensions grow, multiple active cores simultaneously reading from memory can saturate the physical DRAM bus bandwidth. This memory bandwidth bottleneck explains why parallel speedups do not scale linearly at extreme matrix dimensions.

---

7. Experimental Evaluation

7.1 Experimental Testbed and Configuration

The empirical evaluation of MiniTensor was conducted using the integrated benchmark suite (`benchmarks/benchmark.cpp`). The testing environment specifications are summarized in Table 1.

Table 1: Experimental Evaluation Environment

Parameter	Specification
Operating System	Microsoft Windows 11 x64
Compiler	Microsoft Visual C++ (MSVC) 19.51
C++ Standard	ISO C++20 (`cxx_std_20`)
Build Configuration	Release (`/O2 /Ob2 /DNDEBUG`)
Threading Framework	OpenMP Multi-Threading
Hardware Core Count	16 Logical Threads
Cache Line Size	64 Bytes

7.2 Benchmark Methodology and Measurement Protocol

1. **Test Matrices:** Square single-precision floating-point matrices of dimension D in {64, 256, 512, 1024}.
2. **Warmup Protocol:** 1 unmeasured warmup pass was executed prior to timing to prime instruction caches and stabilize CPU clock frequencies.
3. **Repetitions:** Measurements were averaged over 3 consecutive runs using `std::chrono::steady_clock`.
4. **Validation Checksum:** The mathematical accuracy of both blocked and parallel executions was validated against the baseline serial result using a strict absolute tolerance of eps = 10^{-4}:

max_{i,j} |C^{baseline}_{i,j} - C^{test}_{i,j}| <= 10^{-4}

7.3 Empirical Measurements

The empirical execution times and computed speedup factors across three independent benchmark configurations are reported in Tables 2, 3, and 4.

Table 2: Single-Threaded Benchmark (OpenMP Disabled, MSVC Release /O2)

Dimension (N)	Total Elements	Serial Baseline (ms)	Blocked B=64 (ms)	Blocked Speedup	Checksum
64	4,096	0.058 ms	0.058 ms	1.005x	76.738
256	65,536	4.577 ms	4.137 ms	1.106x	320.681

Table 3: Multi-Threaded Benchmark (OpenMP Enabled, 16 Threads, MSVC Release /O2)

Dimension (N)	Total Elements	Serial Baseline ..	Blocked B=64 (ms)	OpenMP (ms)	OpenMP Speedup	Checksum
64	4,096	0.226 ms	0.185 ms	0.041 ms	5.493x	76.738
256	65,536	6.899 ms	6.977 ms	1.281 ms	5.386x	320.681

Table 4: Calibration Benchmark Run Across Extended Dimensions (4 Threads, MSVC Release /O2)

Dimension (N)	Total Elements	Serial Baseline (ms)	Blocked B=64 (ms)	OpenMP (ms)	OpenMP Speedup
64	4,096	0.057 ms	0.067 ms	0.050 ms	1.130x
256	65,536	3.865 ms	3.839 ms	2.381 ms	1.623x
512	262,144	34.627 ms	33.118 ms	24.499 ms	1.413x
1024	1,048,576	238.526 ms	257.955 ms	308.166 ms	0.774x

7.4 Analysis of Performance and Scalability

The experimental data reveals several distinct performance characteristics:

1. **Strong Multi-Threaded Scaling at Medium Dimensions:**

For N = 64 and N = 256 under 16 threads (Table 3), OpenMP multithreading delivers substantial performance gains, achieving **5.493x** and **5.386x** speedups respectively. This demonstrates the efficiency of static row partitioning when the working set fits within aggregated on-chip cache.

2. **Cache Tiling Benefits on Serial Execution:**

At N = 256 (Table 2), cache blocking provides a 10.6% runtime reduction (4.577 ms -> 4.137 ms) by maintaining sub-matrix tiles within L1 data cache.

3. **Memory Bus Saturation at 1024 x 1024:**

In Table 4, at N = 1024, the OpenMP parallel speedup regresses to 0.774x (308.166 ms vs. 238.526 ms serial). This empirical result illustrates a classical systems bottleneck: when four or more cores concurrently process 4 MB matrices, aggregate memory requests exceed the throughput capacity of the CPU memory bus, inducing thread stalling and synchronization overhead.

8. Practical Applications

8.1 Digital Image Processing and Computer Vision

Digital images are natively modeled as rank-3 tensors:

$$I \in \mathbb{R}^{H \times W \times C}$$

where H is image height, W is image width, and C represents color channels (C=3 for RGB, C=4 for RGBA).

MiniTensor's applicability to computer vision is demonstrated in `examples/image_processing.cpp`:

* **Brightness Adjustment:** Translating pixel intensity via coordinate access:

I[y, x, c] <- I[y, x, c] + delta

* **Hadamard Attenuation:** Scaling overall luminance by 50% using tensor multiplication:

I_{dim} = I (x) K, where K[y, x, c] = 0.5

* **Channel Permutation (RGB to BGR):** Swapping color channels via coordinate indexing to interface with standard computer vision formats.

8.2 Dense Neural Network Layers

A standard dense (fully-connected) feedforward layer evaluates:

$$\mathbf{Y} = \mathbf{X}\,\mathbf{W} + \mathbf{B}$$

where X in R^{B x D_{in}} is a batch of inputs, W in R^{D_{in} x D_{out}} is the weight matrix, and B in R^{B x D_{out}} is the bias matrix. MiniTensor directly supports this operation through sequential invocation of `matmul` followed by `operator+`.

8.3 Grid-Based Numerical Simulations

In computational physics and finite-difference methods (e.g., solving the 2D heat equation frac{d u}{d t} = alpha grad^2 u), spatial temperature fields are discretized on a 2D tensor grid. The contiguous storage and stride predictability of MiniTensor facilitate fast local stencil computations.

9. Limitations and Future Research Directions

9.1 Current Architectural Constraints

To maintain technical accuracy, the current limitations of MiniTensor are explicitly identified:

1. **CPU Only:** No GPU acceleration (CUDA, ROCm, or SYCL) is implemented.
2. **No Implicit Element-Wise Broadcasting:** Element-wise binary operations require identical shapes (matrix multiplication supports batch broadcasting).
3. **2D-Only Transposition:** Transposing higher-rank tensors by arbitrary axis permutation is not implemented (users can contract arbitrary axes using `tensordot`).
4. **No Non-Owning Slices/Views:** Operations like `reshape` and `transpose` allocate new owning tensors rather than returning strided non-owning views.
5. **No Automatic Differentiation:** MiniTensor does not provide computational graph tracing or backpropagation.

9.2 Proposed Research Directions

Future development can expand MiniTensor while preserving its lightweight design:

* **Zero-Copy Views and Strided Slicing:** Introducing a lightweight non-owning `TensorView<T>` type modeling sub-regions of existing tensor storage without memory allocation.
* **Multidimensional Transposition / Permutation:** Generalizing axis transposition across arbitrary rank permutations.
* **Explicit Element-Wise Broadcasting Utilities:** Implementing explicit broadcast functions that calculate virtual zero-strides along singleton dimensions.
* **Reduction Operations:** Adding axis-directed reductions such as `sum`, `mean`, and `max`.

10. Conclusion

This paper has presented the engineering design, mathematical foundations, implementation, and empirical evaluation of **MiniTensor**, a lightweight, dependency-free C++20 multidimensional tensor library.

MiniTensor demonstrates that a high-performance, mathematically rigorous tensor container can be implemented without heavy third-party dependencies or excessive template metaprogramming complexity. By combining contiguous row-major memory with precomputed stride vectors, strict RAII value semantics, bounds-checked coordinate validation, and cache-conscious `i-k-j` linear algebra algorithms, MiniTensor bridges the gap between fragile nested vectors and heavyweight industrial frameworks.

Empirical evaluations validate that MiniTensor achieves up to a **5.49x speedup** on multi-threaded workloads, enforces total memory safety across all boundary edge cases, and provides an accessible, robust foundation for systems-level numerical computing.

11. References


```
[1]  I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.
[2]  T. G. Kolda and B. W. Bader, "Tensor decompositions and applications," SIAM Review, vol. 51, no. ...
[3]  B. Stroustrup, The C++ Programming Language, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley,...
[4]  G. Guennebaud, B. Jacob, et al., "Eigen v3," http://eigen.tuxfamily.org, 2010.
[5]  J. Mabille, S. Corlay, and W. Vollprecht, "xtensor: Multi-dimensional arrays with broadcasting an...
[6]  C. F. Van Loan, "The ubiquitous Kronecker product," Journal of Computational and Applied Mathemat...
[7]  D. E. Knuth, The Art of Computer Programming, Volume 1: Fundamental Algorithms, 3rd ed. Reading, ...
[8]  C. R. Harris, K. J. Millman, S. J. van der Walt, et al., "Array programming with NumPy," Nature, ...
[9]  J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Cambr...
[10] ISO/IEC, "Programming Languages -- C++ (ISO/IEC 14882:2020)," International Organization for Stan...
[11] OpenMP Architecture Review Board, "OpenMP Application Programming Interface Version 5.0," Nov. 2018.
[12] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," ACM Transact...
```


12. Comprehensive User Documentation and Integration Guide

This section serves as the complete, practical user manual and engineering integration guide for developers, researchers, and students integrating MiniTensor into their C++ projects.

12.1 Distribution and Download Options

MiniTensor is distributed as a zero-dependency C++20 header library under the MIT License. All official release artifacts are published on GitHub under:

GitHub Releases: https://github.com/yakshaarora2007-sketch/Tensors4Cpp/releases/tag/v1.1.0

Distribution Format	Artifact Name	Size	Recommended Use Case
Amalgamated Single-Header	`minitensor.hpp`	~30 KB	Default choice for 90% of proje..
Modular Archive	`minitensor-v1.1.0.zip`	~31 KB	Recommended for modular project..
Git Repository	`Tensors4Cpp.git`	Full Repo	Recommended for running unit te..

Command-Line Download

Linux / macOS (cURL or wget):

```
curl -LO https://github.com/yakshaarora2007-sketch/Tensors4Cpp/releases/download/v1.1.0/minitensor.hpp
```

or

```
wget https://github.com/yakshaarora2007-sketch/Tensors4Cpp/releases/download/v1.1.0/minitensor.hpp
```

Windows (PowerShell):

```
Invoke-WebRequest -Uri "https://github.com/yakshaarora2007-sketch/Tensors4Cpp/releases/download/v1.1.0...
```

12.2 Integration and Build Setup

Option A: Direct Single-Header Inclusion (Fastest)

Place `minitensor.hpp` directly alongside your source code:

```
my_project/
    minitensor.hpp
    main.cpp
```

In `main.cpp`:

```
#include "minitensor.hpp"
using namespace mt; // or namespace minitensor
```

Option B: Modular Header Inclusion

Extract `minitensor-v1.1.0.zip` so `include/` is in your project search path:

```
my_project/
    include/
        minitensor/
            minitensor.hpp
            tensor.hpp
            indexing.hpp
            linalg.hpp
            operations.hpp
    src/
        main.cpp
```

In your code:

```
#include <minitensor/minitensor.hpp>
// Or include specific domain headers:
#include <minitensor/tensor.hpp>
#include <minitensor/linalg.hpp>
```

Option C: Zero-Download via CMake `FetchContent`

CMake can automatically clone, configure, and link MiniTensor during the build process:

```
cmake_minimum_required(VERSION 3.16)
project(MyTensorApp LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

include(FetchContent)
FetchContent_Declare(
    MiniTensor
    GIT_REPOSITORY https://github.com/yakshaarora2007-sketch/Tensors4Cpp.git
    GIT_TAG        v1.1.0
)
FetchContent_MakeAvailable(MiniTensor)

add_executable(my_app main.cpp)
target_link_libraries(my_app PRIVATE MiniTensor::minitensor)
```

12.3 Complete Multi-Rank Code Walkthrough

The following code snippets illustrate the primary capabilities of MiniTensor across all dimensional ranks.

Example 1: Instantiation, Filling, and Variadic Indexing

```
#include <iostream>
#include "minitensor.hpp"

int main() {
    // Construct a 2x3 single-precision matrix initialized to zero
    mt::Tensor<float> A({2, 3});
```

```
    // Populate elements using variadic coordinate syntax
    A(0, 0) = 1.0f; A(0, 1) = 2.0f; A(0, 2) = 3.0f;
    A(1, 0) = 4.0f; A(1, 1) = 5.0f; A(1, 2) = 6.0f;

    // Construct with uniform fill value
    mt::Tensor<double> B({3, 3}, 42.0);

    // Construct directly from std::initializer_list
    mt::Tensor<int> C({2, 2}, {10, 20, 30, 40});

    std::cout << "A rank: " << A.rank() << ", shape: " << A.shape_str() << "n";
    std::cout << "A(1, 2) = " << A(1, 2) << "n";
    return 0;
}
```

Example 2: Rank 1 Vector Dot Product

```
#include <iostream>
#include "minitensor.hpp"

int main() {
    mt::Tensor<float> u({3}, {1.0f, 2.0f, 3.0f});
    mt::Tensor<float> v({3}, {4.0f, 5.0f, 6.0f});

    // Rank 1 x Rank 1 yields a Rank 0 scalar tensor
    mt::Tensor<float> dot = mt::matmul(u, v);

    std::cout << "Scalar result: " << dot() << "n"; // Output: 32.0f
    return 0;
}
```

Example 3: Rank 2 Matrix Multiplication & Cache-Blocked Matmul

```
#include <iostream>
#include "minitensor.hpp"

int main() {
    // 2x3 matrix
    mt::Tensor<float> A({2, 3}, {
        1.0f, 2.0f, 3.0f,
        4.0f, 5.0f, 6.0f
    });
    // 3x2 matrix
    mt::Tensor<float> B({3, 2}, {
        7.0f, 8.0f,
        9.0f, 1.0f,
        2.0f, 3.0f
    });

    // Baseline serial matrix multiplication
    mt::Tensor<float> C = mt::matmul(A, B);

    // Cache-blocked multiplication with block size B=64
    mt::Tensor<float> C_blocked = mt::matmul_blocked(A, B, 64);

    std::cout << "C shape: " << C.shape_str() << "n"; // [2, 2]
    std::cout << "C(0, 0) = " << C(0, 0) << "n";     // 31.0f
    return 0;
}
```

Example 4: Rank 3 Batched Matrix Multiplication

```
#include <iostream>
#include "minitensor.hpp"

int main() {
    // Batch of two 2x3 matrices (Shape: [2, 2, 3])
    mt::Tensor<float> batchA({2, 2, 3}, {
```

```
        1, 2, 3,  4, 5, 6,     // Batch slice 0
        7, 8, 9,  1, 2, 3      // Batch slice 1
    });

    // Batch of two 3x2 matrices (Shape: [2, 3, 2])
    mt::Tensor<float> batchB({2, 3, 2}, {
        9, 8,  7, 6,  5, 4,    // Batch slice 0
        3, 2,  1, 0, -1, -2    // Batch slice 1
    });

    // Batched matrix multiplication: evaluates each slice independently
    mt::Tensor<float> batchC = mt::matmul(batchA, batchB);

    std::cout << "Result shape: " << batchC.shape_str() << "n"; // [2, 2, 2]
    return 0;
}
```

Example 5: Batch Dimension Broadcasting

```
#include <iostream>
#include "minitensor.hpp"

int main() {
    // Batched input tensor: Shape [4, 10, 32] (Batch size 4, 10 tokens, 32 dims)
    mt::Tensor<float> X({4, 10, 32}, 1.0f);

    // Single linear layer projection matrix: Shape [32, 64]
    mt::Tensor<float> W({32, 64}, 0.5f);

    // Batch broadcasting automatically aligns W across all 4 slices
    mt::Tensor<float> Y = mt::matmul(X, W);

    std::cout << "Output shape: " << Y.shape_str() << "n"; // [4, 10, 64]
    return 0;
}
```

Example 6: Arbitrary-Axis Tensor Contraction (`tensordot`)

```
#include <iostream>
#include "minitensor.hpp"

int main() {
    // 3D tensor: Shape [2, 3, 4]
    mt::Tensor<float> A({2, 3, 4}, 1.0f);
    // 3D tensor: Shape [4, 5, 6]
    mt::Tensor<float> B({4, 5, 6}, 2.0f);

    // Contract axis 2 of A (dimension 4) with axis 0 of B (dimension 4)
    mt::Tensor<float> C = mt::tensordot(A, B, {2}, {0});

    // Resulting outer shape: [2, 3, 5, 6]
    std::cout << "Contracted shape: " << C.shape_str() << "n";
    return 0;
}
```

Example 7: Reshaping and Transposition

```
#include <iostream>
#include "minitensor.hpp"

int main() {
    mt::Tensor<float> M({2, 3}, {1, 2, 3, 4, 5, 6});

    // Materialized matrix transpose: [2, 3] -> [3, 2]
    mt::Tensor<float> MT = M.transpose();

    // Materialized reshape: [2, 3] -> [6] or [1, 6]
    mt::Tensor<float> flat = M.reshape({6});
```

```
    std::cout << "MT shape: " << MT.shape_str() << ", flat shape: " << flat.shape_str() << "n";
    return 0;
}
```

12.4 Compiler Commands and Flags

MiniTensor requires a standard C++20 compiler. No auxiliary libraries or link flags are needed for serial execution.

GCC (Linux, MinGW-w64):

```
# Serial compilation:
g++ -std=c++20 -O3 -I. main.cpp -o app

# With OpenMP multithreading acceleration:
g++ -std=c++20 -O3 -fopenmp -DMINITENSOR_USE_OPENMP -I. main.cpp -o app
```

Clang (macOS, Linux):

```
# Serial compilation:
clang++ -std=c++20 -O3 -I. main.cpp -o app

# With OpenMP multithreading acceleration:
clang++ -std=c++20 -O3 -fopenmp -DMINITENSOR_USE_OPENMP -I. main.cpp -o app
```

Microsoft Visual C++ (MSVC / Visual Studio):

```
:: Serial compilation:
cl /std:c++20 /O2 /EHsc /I. main.cpp /Fe:app.exe

:: With OpenMP multithreading acceleration:
cl /std:c++20 /O2 /EHsc /openmp /DMINITENSOR_USE_OPENMP /I. main.cpp /Fe:app.exe
```

12.5 API Reference and Method Cheat Sheet

Category	Method / Signature	Description	Complexity
Construction	`Tensor(shape, fill_value=0)`	Allocates owning tensor of give..	O(N)
	`Tensor(shape, values_vector)`	Constructs tensor and moves/cop..	O(N)
	`Tensor(shape, initializer_list)`	Constructs tensor from braced l..	O(N)
Shape Queries	`rank()`	Returns number of dimensions (K).	O(1)
	`shape()`	Returns `const std::vector<size..	O(1)
	`size()`	Returns total element count (N ..	O(1)
	`empty()`	Returns `true` if N == 0.	O(1)
	`strides()`	Returns precomputed row-major s..	O(1)
	`shape_str()`	Returns formatted shape string ..	O(K)
Coordinate Access	`operator()(indices...)`	Variadic coordinate lookup (e.g..	O(K)
	`operator()()`	Scalar lookup for Rank 0 tensors.	O(1)
	`at(indices)`	Bounds-checked coordinate looku..	O(K)
	`set(indices, value)`	Explicit coordinate element ass..	O(K)
Transformations	`reshape(new_shape)`	Returns new tensor with new sha..	O(N)
	`transpose()`	Returns materialized transpose ..	O(M * N)
Element-Wise Ops	`+`, `-`, `*`	Point-wise binary arithmetic be..	O(N)
	`==`, `!=`	Value and shape equality compar..	O(N)
Linear Algebra	`matmul(A, B)`	Arbitrary-rank tensor multiplic..	O(B * M * K * N)
	`matmul_blocked(A, B, B_size=64)`	Cache-tiled matrix and batched ..	O(B * M * K * N)
	`matmul_parallel(A, B)`	OpenMP multi-threaded parallel ..	O(frac{B * M * K * N}{P})
	`tensordot(A, B, axes_a, axes_b)`	General tensor contraction alon..	O(V * K_{contract})
Memory Access	`data()`	Direct pointer access to contig..	O(1)
	`raw_data()`	Returns underlying `std::vector..	O(1)